Author Guidelines for OpenChapters Contributions

Anthony D. Rollett & Marc De Graef

May 29, 2026

Note: OpenChapters requires TeXLive 2026 to properly function!

1 OpenChapters: Introductory Remarks

1.1 What is it?

The OpenChapters project is an open source project for the automated generation of textbook-style PDF files and/or HTML web collections, starting from a curated collection of individual user contributed chapters. The chapters are written by various authors using the LaTeX guidelines described in this document and are contributed to a GitHub-hosted chapter repository under the Creative Commons CC BY-NC-SA 4.0 License. Via a web portal, an instructor can select from the available chapters those that are needed for a particular course; the web server then combines those chapters along with supporting or foundational chapters into a fully formatted document or a collection of web pages that can then be downloaded and distributed as needed.

1.2 What is our thinking behind this?

Many faculty members, in particular those who have been teaching college level courses for a while, have lecture notes, handouts, slide decks, and so on, along with vague plans to, one day, turn those into a full blown textbook. At the same time, students, both at the undergraduate and graduate level, increasingly expect course notes to be made available for free, typically in PDF form. Many classic textbooks are now available in electronic form (e.g., e-books) through university library agreements with publishing houses. There is substantial pressure on publishers to modernize the classical textbook model because hard cover books are becoming increasingly (and excessively) expensive for students to purchase. With the recent rise of on-line AI interfaces there is little incentive left for faculty members to publish their course notes in text book form: (1) most publishers expect authors to do most or all of the typesetting and the creation of illustrations, so that writing a text book is a truly monumental task; and (2), royalties on such books are very low, basically removing any potential financial incentives. The "return on investment" metric for textbooks is hence depressingly low, except perhaps for general undergraduate books for introductory courses in mathematics, physics, etc. for which there is a large potential audience; a quick search for such books, however, shows that this market is rather saturated, with many competing textbooks, and it is not clear yet how the rise in AI interfaces will affect those textbooks. Thus, many instructors are faced with a multi-facetted dilemma:

- should students be told to buy expensive textbooks from established publishing houses, even when only a small portion of such a book is actually used in class? [and buying used textbooks is not much cheaper these days]
- does it really make sense for students to rent textbooks?
- should the instructors develop and sell their course notes (i.e., self-publish)? Or should they make them available for free?
- . . . [fill in your own concerns and questions]

In an ideal world, an instructor should be able to assemble a textbook that is perfectly suited to the course's needs; the components of such a book could consist of the instructor's own course notes, combined with chapters from other authors. What we need is a reconfigurable collection of chapters that can be made into just the book the instructor needs. And the following year, the collection will have new chapters in it, and existing chapters may have been updated, so that the PDF file generated from the collection of chapters becomes an up-to-date and fully customizable "one-of-a-kind textbook", free to the student and instructor alike.

1.3 Where can you find everything?

All of the content of this project is open source and falls under a Creative Commons license. You can find the source by searching for the OpenChapters organization on GitHub. The top folder contains several repositories:

- OpenChapters: the main repository for all contributed chapters;
- OCwebdesign: repository for the OpenChapters book-building web interface;
- OCchaptertemplate: repository with an example chapter;
- BookBuilder: an early attempt at automating the book typesetting process by running everything on GitHub servers (no longer in development).

As with any open source project on GitHub, you can fork the complete OpenChapters repository into your own GitHub account, and then clone the repository onto your own computer. If you are interested in contributing one or more chapters you can fork and clone the OCchaptertemplate and then edit your own chapter following some guidelines in the various files from this repo.

If you wish to assemble your own text book, then you can do this simply by using the following web site:

`https://openchapters.materials.cmu.edu  [active for beta-testing]`

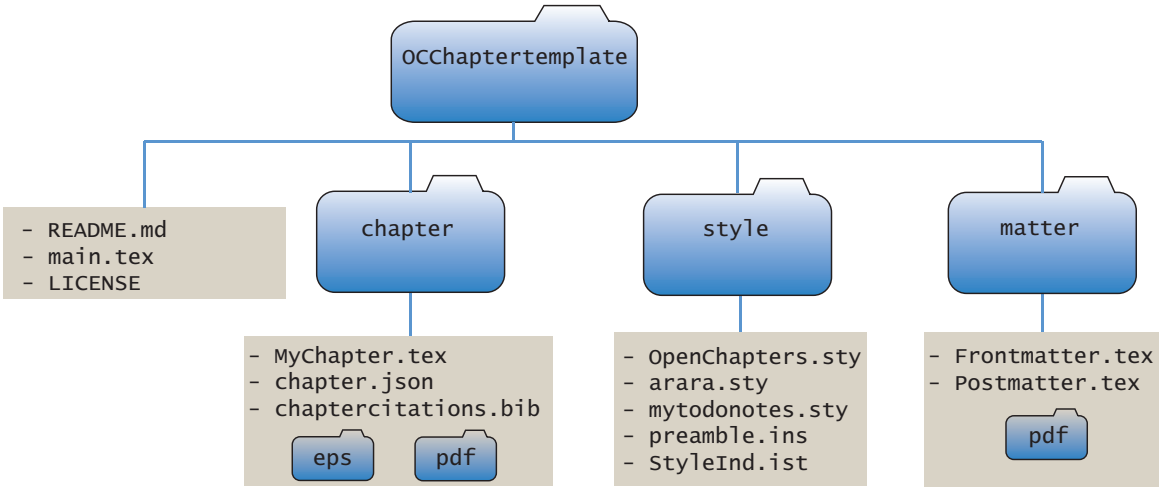
You will need to sign in (or obtain a username/password if this is your first access attempt), and then you can select the chapters you want, hit the Build PDF or Build HTML button and wait for a download link to appear in your email box.

More detailed instructions about the functionality of the web interface are available in the User Guide, accessible as a MarkDown file from the main menu.

2 Basic Chapter Formatting

An author who wishes to contribute a chapter to the OpenChapters repository should first consult the formatting instructions in this section. OpenChapters uses a series of LaTeX packages to achieve a uniform layout and a full installation of TeXLive 2026 is required. From the OCchaptertemplate repository, the prospective author can clone a series of folders that form the basic layout to create a new chapter. Below is a description of the most important components of the OCchaptertemplate package.

2.1 Overall Folder Structure



The OpenChapters chapter template file will unzip into the folder structure shown above. The top folder has a series of files and three subfolders:

- README.md: This is the top level markdown file that serves as a descriptor for the git repository.
- main.tex: This is the top level LaTeX source file that controls the typesetting.
- LICENSE: Creative Commons license file.
- chapter folder: For each chapter there is a source file MyChapter.tex and a bibliography file, chaptercitations.bib. During a build, the chapter's .aux file will be stored in this folder. Figures for each chapter should be stored in the pdf subfolder and should be formatted as a .pdf file; auxiliary figure files (.tiff, .png, .ai, etc.) should be stored in a separate folder that is not committed to the repository (i.e., place the folder path in the .gitignore file). Each chapter also has a required chapter.json file that contains important

fields for the typesetting engine in the full OpenChapters distribution.

- style folder: All the necessary .sty files and a collection of user-defined commands in preamble.ins can be found here.
- matter folder: Frontmatter.tex generates the title page, copyright notice, a list of chapter authors and affiliations, and the table of contents. Postmatter.tex takes care of the bibliography and the index. These files should not need any edits. The pdf folder should contain a chapter header image (described below).

2.2 Main Source File main.tex

The typesetting process is controlled by the arara package, a Java-based automation tool for TeX and LaTeX document compilation that uses explicit instructions (directives) placed directly within the source code to determine the build process. These commands are commented out with LaTeX comment symbols (%) and should not be altered.

Then the file contains some general typesetting comments:

Document preamble

```
% ######################################################################
% OpenChapters - Standalone Chapter Template
%
% This main.tex file typesets a single chapter for preview and editing.
% To compile, run:  arara -w main.tex
% (or use your editor's build button if it supports arara)
%
% Manual compilation alternative (if arara is not available):
%   pdflatex -shell-escape main
%   makeindex -s style/StyleInd.ist main
%   biber main
%   pdflatex -shell-escape main
%   pdflatex -shell-escape main
%
% Licensed under the Creative Commons CC BY-NC-SA 4.0 License.
% ######################################################################
```

Note that manual compilation instructions are also provided, in case the local TeXLive installation does not contain the arara package. Then we start the document using the standard book class, but heavily customized in the OpenChapters.sty style file. User-defined commands should be added to the preamble.ins file (please do not remove or edit any existing commands; also, please pay attention to the comments below about the use of the \newcommand command).

Document preamble

```
\documentclass[11pt,fleqn,A4paper]{book}

% Load the OpenChapters style from the style/ directory
\makeatletter
\def\input@path{{style/}}
\makeatother
\usepackage{OpenChapters}
\usepackage{arara}

% Figures are stored in chapter/pdf/
\renewcommand{\chaptergraphicspath}{chapter/pdf/}
\renewcommand{\graphicspath}{chapter/pdf/}
\newcommand{\isHTML}{No}
\newcommand{\isEPUB}{No}

% User-defined commands from the style package
\input{style/preamble.ins}

% Bibliography for this chapter
\addbibresource{chapter/chaptercitations.bib}

% OpenChapters macro
\newcommand{\OC}{\textsf{OpenChapters}}
```

Then we start the actual document:

Front matter

```
\begin{document}

% Simplified front matter for standalone chapter preview
\input{matter/Frontmatter}

% Open the authors file for writing
\immediate\openout\authorfile=\jobname-authors.aux%

% ======================================================================
% YOUR CHAPTER - change the path below to match your chapter filename
% Note that multiple chapters are allowed; simply add more include lines
% as needed.
% ======================================================================
\include{chapter/MyChapter}

% Post matter (bibliography and index)
\input{matter/Postmatter}

\end{document}
```

The Frontmatter.tex file takes care of the title and copyright pages, the list of

chapter authors with affiliations, and the table of contents. The file that contains the author information is then opened so that the present typesetting run can write to it; this must be done *after* the Frontmatter.tex file is read.

Then we have one or more \include commands, one for each potential chapter. Note that document parts should not be used since parts are created during the book building step on the OpenChapters web site. The main.tex file concludes with the citations and index which are handled in the Postmatter.tex file.

2.3 The MyChapter.tex and chapter.json files

To facilitate integration of a new chapter into the full-scale OpenChapters project, each chapter must provide information for two different processes: the web interface and the book build process. Information for the web user interface is contained in a small chapter.json file, whereas the broader LaTeX typesetting information is located at the start of an individual chapter source file. Note that there is some unavoidable duplication of information.

The chapter.json file

Information for the OpenChapters web interface is contained in a chapter.json file; the general layout of this file should not be changed but individual fields should be filled in by the author(s) before a chapter pull request is submitted to the project editors.

The chapter.json file

```
{
  "title": "REPLACE: Your Chapter Title",
  "authors": [
    "REPLACE: Your Name"
  ],
  "description": "REPLACE: A brief description of this chapter.",
  "toc": [
    "Introduction",
    "Methods",
    "Worked Examples"
  ],
  "entry_file": "MyChapter.tex",
  "cover_image": "cover.png",
  "keywords": [],
  "chapter_type": "topical",
  "chabbr": "MYCHAP",
  "depends_on": [],
  "discipline": "mse",
  "published": false,
  "author_urls": "http://some_link.html"
}
```

Note that multiple chapter authors are possible. The toc field should contain all the main section titles from your chapter. The cover_image parameter should not be changed; this will be used by the book building code to generate a thumbnail for your chapter. For the chapter_type, please select either "topical" or "foundational". The chabbr string should be uppercase, with six characters as an abbreviation for your chapter title. Topical chapters can depend on one or more Foundational chapters; the depends_on field should contain all the chabbr strings for foundational chapters your chapter depends on. The other parameters in the json file can be left unchanged and will be adjusted as needed by the editors.

As an example, we provide here the contents of the chapter.json file for the Crystallography chapter:

chapter.json example

```
{
  "title": "Crystallographic Computations",
  "authors": [
    "Marc De Graef"
  ],
  "description": "Basic crystallographic computations",
  "toc": [
    "Two Views of the Real World",
    "Crystallographic Computations in a Bravais Lattice",
    "Transforming between Direct and Reciprocal Spaces",
    "Conversion to a Standard Cartesian Reference Frame",
    "Other Useful Crystallographic Computations"
  ],
  "entry_file": "Crystallography.tex",
  "cover_image": "cover.png",
  "keywords": [
  "Metric tensor",
  "Direct space",
  "Lattice geometry"
  ],
  "chapter_type": "topical",
  "chabbr": "BASCRY",
  "depends_on": [
    "LINALG",
    "TENSOR"
  ],
  "published": true,
  "discipline": "mse",
  "author_urls": {
  "Marc De Graef": "https://www.mse.engineering.cmu.edu/directory/[...].html"
  }
}
```

If an author makes changes to this file in their local repository, the modified file will need to be updated in the main repository by means of a pull request to

the editors.

Chapter Header Components

To make it possible to automatically generate a PDF file from a repository of individual chapters, each chapter has a few required components that need to be defined at the start of the source file. The chapter template file, MyChapter.tex, which is the starting point for any new chapter, has the following mandatory commands after the copyright statement:

Mandatory chapter commands

```
% ######################################################################
% AUTHOR SETUP - Edit the lines below
% ######################################################################

% Uncomment the next line ONLY if this is a foundational chapter:
%\corechapter{Yes}

% Your name and affiliation (appears below the chapter title):
\OCchapterauthor{REPLACE: Your Name, Your University}

% Chapter abbreviation: choose a UNIQUE 6-character uppercase code.
% This is used as a prefix for all labels in this chapter.
% Examples: LINALG, TENSOR, NUMSYS, PROJTS
\renewcommand{\chabbr}{MYCHAP}

% Chapter header image (2480 x 1240 pixels, 300 dpi, RGB format).
% Replace \noheaderimage with your header filename (e.g., MYCHAPheader.pdf).
% The file should be placed in chapter/pdf/.
% Note that \graphicspath is defined in the main.tex file.
%\chapterimage{\graphicspath MYCHAPheader.pdf}
\chapterimage{\noheaderimage}

% ######################################################################
% CHAPTER TITLE - Edit the title below
% ######################################################################
\chapter{My Chapter Title}\label{\chabbr:ch:MyChapter}

% Register author information (repeat for each co-author)
\writeauthor{\chabbr:ch:MyChapter}{My Chapter Title}{REPLACE: Last}
{REPLACE: First}{REPLACE: Department}{REPLACE: University}
{REPLACE: email@example.com}{REPLACE: https://homepage.example.com}

% ######################################################################
% LEARNING OBJECTIVES
% ######################################################################
% Each chapter begins with Learning Objectives.  Each objective should:
% - Contain an active verb (Learn, Understand, Derive, Apply, etc.)
% - Link to the relevant section using the \chabbr:sec:label format
% - Use the OCBurntOrange color for the reference
\begin{learningobjectives}
{\begin{itemize}
    \item[{\color{OCBurntOrange}\ref{\chabbr:sec:introduction}:}]
       Understand the basic concepts ...
```

Mandatory chapter commands (ctd.)

```
44      \item[{\color{OCBurntOrange}\ref{\chabbr:sec:methods}:}]
45         Learn the methods used for ...
46      \item[{\color{OCBurntOrange}\ref{\chabbr:sec:examples}:}]
47         Apply the concepts to ...
48  \end{itemize}}
49  \end{learningobjectives}
```

- corechapter: If the chapter is a foundational or core chapter, then this line needs to be uncommented, otherwise it must be left commented out.
- OCchapterauthor: This must be set to a brief single line string containing the author name(s), and a short affiliation; this will be typeset just below the chapter title.
- chabbr: All labels for sections, subsections, equations, figures, tables, etc., need to have a unique chapter identifier so that the OpenChapters typesetting engine can figure out all the dependencies between individual chapters. This identifier is a six-character string, in upper-case form, that abbreviates the chapter title. So, for a *Basic Crystallography* chapter, the string may be defined as BASCRY.
- chapterimage: The chapter title and the brief chapter author info will be superimposed onto a unique header image that must be provided by the author(s). The image should be in *.pdf format with 2480 × 1240 pixels at 300 dpi, RGB color format. If no such image is available, then the entry should have the \noheaderimage command as a parameter.
- chapter: this is where the actual chapter command is executed. The full title should be entered as an argument, and the command should be followed by a corresponding label.
- writeauthor: this is a command with 8 arguments:
  1. the full chapter label, consisting of the 6-letter abbreviation followed by ":ch:", and the one-word chapter title (exactly what went into the \label command on the previous line);
  2. the full chapter title;
  3. author last name;
  4. author first name;
  5. author department;
  6. author institution;
  7. email address (no need to escape the @ sign);
  8. the author's full home page URL.

This command should be repeated for every author in case of a multi-author chapter.

- Each chapter header ends with the Learning Objectives. There is a pre-defined learningobjectives environment that should be called with the standard \begin and \end commands. Between those commands, there should be an itemize environment surrounded by curly braces { } (do not forget them because the typesetting process will throw an error...). Ideally, there would be one entry per section of the chapter. In each entry, the \item command has an argument between square brackets; the argument must be enclosed in curly braces and sets the color to OCBurntOrange and then calls the section label which replaces the usual bullet symbol. Then there should be a very brief statement about that section; this statement should contain a verb to make it an active statement; note that hyphenation is turned off in this environment. Fig. 1 shows an example chapter header for a foundational chapter on number systems.

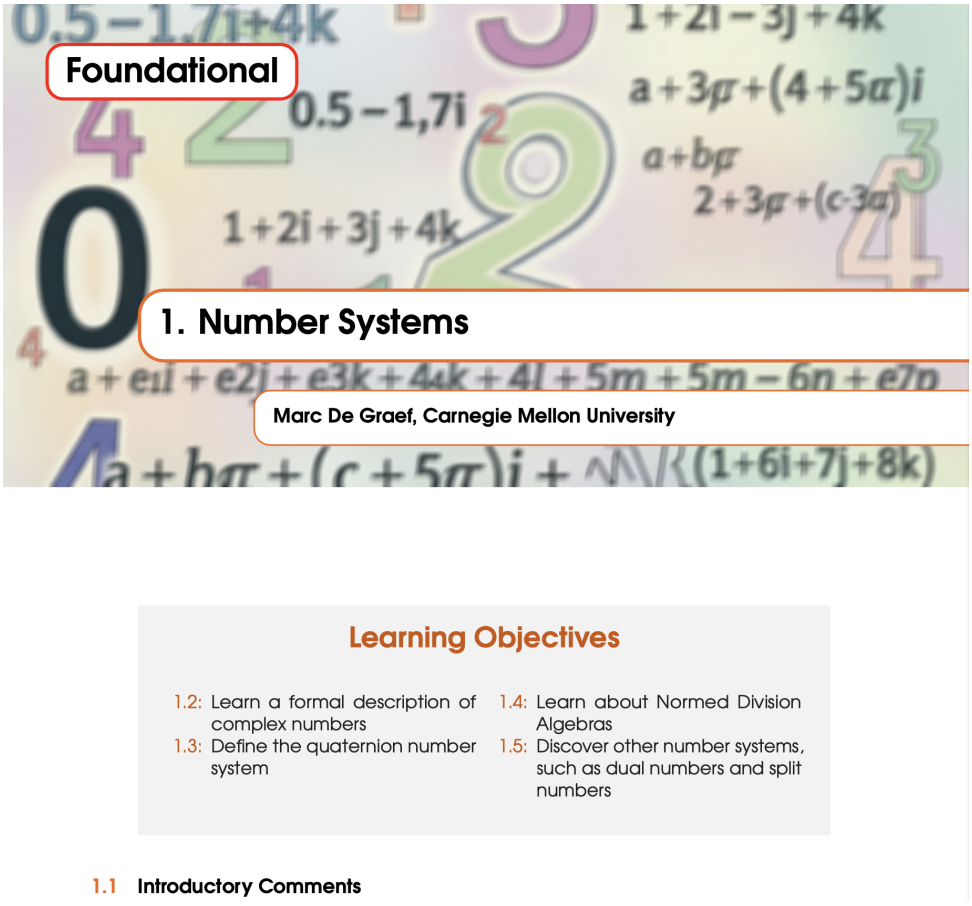


Figure 1: Example chapter header showing the various components that are defined with commands above.

2.4 Labels

As stated above, all user-defined labels will need to have the \chabbr string as part of any \label command; thus, to label a figure with the "tree" label, the command \label{\chabbr:fig:tree} must be used, and, inside that chapter, this label can be referenced with \ref{\chabbr:fig:tree}. From a different chapter, this figure can be referenced by explicitly using the correct 6-letter string; so, if the "tree" figure label was defined in a chapter with label \chabbr=BASCRY, and you wish to reference this figure from another chapter, you must use the explicit label \ref{BASCRY:fig:tree}. To streamline the labeling of various components, we urge the author to use the following conventions:

- Abbreviate the environments as follows: chapter → ch; section → sec; subsection → ssec; subsubsection → sssec; figure → fig; table → tb; equation → eq, etc.;
- Use a colon to separate the environment identifier from the actual unique label name, for instance \chabbr:fig:tree or \chabbr:sec:introduction;
- *Do not use numerals as part of any labels*, i.e., do not use \chabbr:fig:f1 or \chabbr:tab:tb2; instead, let LaTeX take care of all the numbering for you;
- *Do not use spaces or underscore characters in any label definition.*

Adhering to this convention will make it easier to properly reference items in a volume with potentially many chapters that tightly interdepend on each other.

2.5 Foundational or Topical Chapters?

The OpenChapters project currently recognizes two different chapter categories:

- Foundational (or core) chapters;
- Topical chapters.

A topical chapter is a chapter that covers an important topic in great detail; for instance, the use of thermodynamic and crystallographic symmetry to describe material property tensors, or the determination of the orientation distribution function from pole figures. A foundational chapter on the other hand is a chapter that contains basic information about one or more topics. For instance, let's say that Author A writes a topical chapter on material textures, Author B works on a chapter that covers different orientation representations, and author C has a chapter on visual representation of point groups. There are several common areas to these three topics, in particular the use of Euler angles and stereographic projections. Instead of each topical chapter having a section defining stereographic projections, it makes more sense to have a single chapter defining this projection, listing its properties and providing examples; this would be a core or foundational

chapter that all three authors can then refer to for definitions. This approach separates the basic material from the more advanced material that makes use of the basics. Using the labeling format described above, and assuming that the stereographic projection chapter goes by the abbreviation STPROJ, all three authors A, B, and C can refer to equations, figures, etc. from this chapter in their own chapter, simply by using the standard \ref{} command with a full label string of the type STPROJ:fig:3Dview. The OpenChapters web site makes available a PDF file for each foundational chapter; this file is compiled using the showlabels package, so that all defined labels will show up in the file. This facilitates the referencing of core material from the topical chapters.

Note that the author should list all the dependencies on Foundational chapters in the depends_on field in the chapter.json file, so that the web interface can properly include all necessary chapters.

While topical chapters can refer to items in other topical chapters as well as foundational chapters, the opposite is not true; *a foundational chapter should never reference any topical chapters, only other foundational chapter content.* The simple reason for this is that we want to avoid circular dependencies.

2.6 Figures and Tables

Due to the Creative Commons license used for this project, ***all figures used in any of the contributed chapters must be original figures free of existing copyright claims***. We wish to avoid any potential copyright issues by simply not accepting any figure that already falls under anybody else's copyright. We realize that this imposes more demands on the author but it is the only way in which a project like OpenChapters can be successful. The only exception will be for figures that already fall under one of the Creative Commons licenses; for such figures, it is the author's responsibility to state the relevant figure copyright status as well as the original source citation in the figure caption.

All figures should be submitted in *.pdf format and must be fully editable, i.e., the editors should be able to change the font of the lettering used on diagrams, or the font used for a scale bar on an image. If available, original Adobe Illustrator files (*.ai) or similar would be most welcome. Figures inside PowerPoint files are generally of too low a quality and will not be accepted.

For tables, we use the standard LaTeX approach in terms of formatting.

2.7 PDF vs. HTML output

The web back-end of this project employs the arara package to control the typesetting process. For regular PDF output, this means that the pdflatex program is called multiple times, along with makeindex for the index and biber for the bibliography. For HTML output, on the other hand, the typesetting engine employs the lwarp package, with a single call to pdflatex followed by multiple calls to lwarpmk. The lwarp package does not support all the packages available in the standard TeXLive 2026 distribution, which means that the author should be mindful in particular of restrictions with respect to tables.

The main.tex file contains two special commands:

Document preamble

```
\newcommand{\isHTML}{No}
\newcommand{\isEPUB}{No}
```

They should be left unchanged for PDF typesetting, which is the default typesetting mode to be used by authors; they are used by the web back-end to generate the HTML version of the chapter(s) (and in the future potentially an EPUB version). The most important lwarp limitations pertain to the tabular environment used to generate tables. Any table in which individual cells contain special typesetting (e.g., colored text or equations) should be handled using the \isHTML command as illustrated here:

Document preamble

```
\begin{table}[h]
\centering\leavevmode
\ifthenelse{\equal{\isHTML}{Yes}}{% we load a pdf image of the table
  % since lwarp may have issues with the actual table
  \includegraphics[width=0.9\textwidth]{\graphicspath table-image.pdf}
}{%else we do the actual table with standard formatting
  \begin{tabular}{}
  ...
  \end{tabular}}
  \caption{...}
\end{table}
```

This work-around requires that the author provide a pdf image with, essentially, a screen shot of the table as typeset in the PDF version of the chapter. The

screen shot should have sufficient resolution (e.g., better than 300 dpi) so that it renders well as an image in the HTML version of the chapter. As an example in an actual chapter, look for the comment line "% AUTHORGUIDELINES EXAMPLE TABLE" in the Symmetry.tex source file.

2.8 User-defined commands

All user-defined special commands and/or environments should be collected in the preamble.ins file in the main build folder. To avoid naming conflicts, it would be good practice to pre-pend the author's initials in all-caps to any new command that they define. New environments will need to be extensively tested for compatibility with the HTML version of the typesetting style files; thus, the editors reserve the right to modify any user-defined environment to ensure full compatibility with both PDF and HTML output.

2.9 Color palettes

The OpenChapters package uses as its default colors all the colors from the CMU color palette. Each color is named by the official CMU name, preceded by the letters "OC". There are no restrictions on the use of colors, and authors may pick any color they like; the ones below are pre-defined and form a consistent color palette, but their use is completely optional. Here are the available colors:

Core colors:

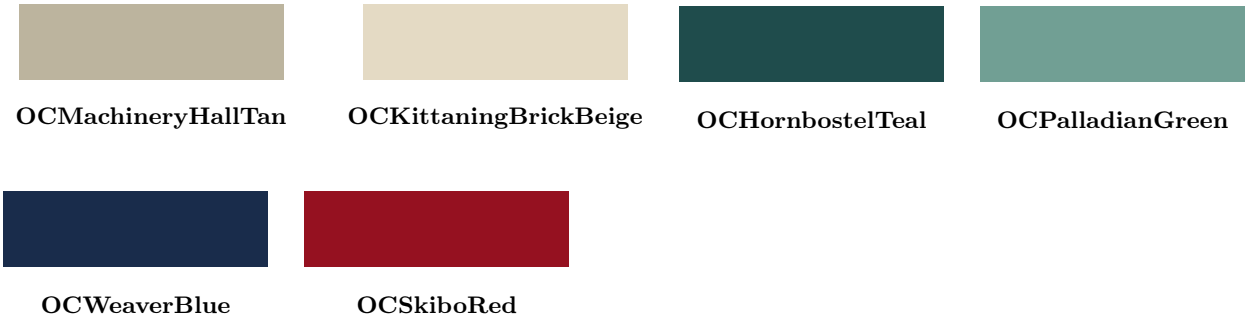
OCCarnegieRed **OCSteelGray** **OCIronGray** **OCBlack**

Secondary colors, Tartan Palette:

OCScotsRose **OCGoldThread** **OCGreenThread** **OCTealThread**

OCBlueThread **OCHighlandsSkyBlue**

Secondary colors, Campus Palette:



In addition to `OCwhite`, for which RGB=(255,255,255), there are currently three additional colors used for the Table of Contents entries:

2.10 Citations

The OpenChapters collection is meant to be foremost of an educational nature, i.e., chapter content is ideally accessible at the undergraduate level, with clear (new!) illustrations, transparent explanations, and connectivity between chapters. As such, there is no need to be exhaustive in citing the relevant literature; these are not intended to be review chapters with hundreds of citations per chapter. Instead, citations should be carefully selected to:

- provide historical background for important topics;
- introduce the reader to seminal papers or books in the field;
- illustrate relevance to everyday applications.

Citation typesetting

The OpenChapters typesetting engine uses biblatex to handle citations, and the biber program to typeset them; the following biblatex package options are used:

```
style=ext-numeric,
citestyle=numeric-comp,
sorting=none,
autopunct=true,
autolang=hyphen,
```

```
hyperref=true,
abbreviate=false,
backref=false,
backend=biber,
autocite=superscript,
maxnames=99,
articlein=false,
casechanger=latex2e
```

The author is urged to use the following convention for citations:

- all citations should have a DOI field;
- all citation keys should be of the form lastnameYEARa, e.g., smith1995a

Note that the key is all lower-case, contains only the last name of the first author of the paper, followed by the publication year using four digits, and the letter *a*, *b*, etc, to differentiate between multiple citations from the same author name and year. This formatting of the citation key makes it easy for the editors to find duplicate keys that are used in different chapters.

Do not use spaces or underscores in citation keys!

Citations of AI-generated content

The rapid advances of large language models, including their ability to generate images and other media content, raise the question of how to properly cite the use of such models, as well as questions of ownership. This is a rapidly evolving area, and most publishing companies are developing their own guidelines for scientific publications. On a broader level, the Author's Alliance offers some responses to questions of ownership of AI generated content; we suggest that authors who wish to include AI-generated media in their chapter follow these guidelines as a starting point. Summarizing the most important points:

- If an AI-generated figure is solely based on a figure prompt by the author, and nothing else, then that figure is generally considered to be uncopyrightable. In that case, a simple statement in the figure caption would be sufficient; for instance: *(AI-generated image via Google's Gemini, version 3.1)*. We also suggest that the prompt itself be included in the LaTeX source file as a comment, inserted immediately after the figure.
- If an AI-generated figure is based on a copyrighted figure from a book or other publication, and *that figure was uploaded as part of the prompt* (perhaps to give a hint as to what the new figure might look like), then it is unclear at this point in time[1] what the status of the new figure is. In most cases there is

[1] May 29, 2026

no established case-law that provides a clear-cut interpretation or guideline. For authors contributing to the OpenChapters project, we suggest that only figures that were published under a Creative Commons license be used to guide an LLM to generate a new figure; this must be clearly stated in the figure caption along with a link to the original media source.

Given the nature of the OpenChapters project as an open source project under a Creative Commons license, contributing authors should be conservative in their inclusion of AI-generated material, and always err on the side of "overdoing it" when it comes to citing LLM origins. The entire publishing world is struggling to keep up with this rapidly evolving AI field, so authors should be aware that OpenChapters guidelines may change in response to this evolution.

2.11 Index entries

The OpenChapters typesetting engine automatically generates an index using the standard makeindex program, and is "prettyfied" using the entries in the StyleInd.ist file. In the main text, the author can use the default \index command, or, if the word should also appear in the text, it can be simultaneously italicized and placed in the index by means of the \indexit command.

2.12 Predefined boxes

Often an author will want to highlight a concept, or provide a piece of background information, and so on. Several highlight box types are predefined in the arara.sty package and can be called as needed. The standard commands to be entered in the source file are as follows:

```
\begin{messagebox}{HEADER}{COLOR}{BOXTYPE}{HEADERCOLOR}
    box content ... basically anything typeset in LaTeX.
\end{messagebox}
\begin{codebox}{HEADER}{COLOR}{BOXTYPE}{HEADERCOLOR}
    box content ... basically anything typeset in LaTeX.
\end{codebox}
\begin{ncodebox}{HEADER}{COLOR}{BOXTYPE}{HEADERCOLOR}{CODENAME}
    box content ... basically anything typeset in LaTeX.
\end{ncodebox}
```

Inside each box example below you will find the definition of the parameters used to typeset the box. Note that for the `BOXTYPE` parameter the following options are available: `\icattention`; `\icerror`; `\ichelp`; `\icinfo`; `\icnote`;

\icok. This parameter determines which icon is used in the upper left corner of the box. You can experiment with these and the available boxes to generate your own boxes; we do suggest that the colors used belong to the extended CMU color palette described earlier, but that is just a suggestion. Note that boxes are automatically split across page breaks if necessary.

Test box

```
HEADER= Test box
COLOR= OCBlueThread
BOXTYPE= \icinfo
HEADERCOLOR= OCblack
```

Another test box

```
HEADER= Another test box
COLOR= OCpurple
BOXTYPE= \icattention
HEADERCOLOR= OCwhite
```

An OK box

```
HEADER= An OK box
COLOR= OCBlueThread
BOXTYPE= \icok
HEADERCOLOR= OCwhite
```

The contents of a codebox are automatically placed in verbatim mode (so that line indents are maintained in the output) and will have line numbers:

A code box

```
1       HEADER= A code box
2            COLOR= OCSteelGray
3       BOXTYPE= \icnote
4       HEADERCOLOR= OCwhite
```

A named code box

function f(x)

```
HEADER= A named code box
     COLOR= OCIronGray
BOXTYPE= \icnote
HEADERCOLOR= OCwhite
CODENAME= function f(x)
```

Alternatively, one could use any one of the available source code formatting packages and place the (pseudo) code in one of these boxes. Packages like `listings` or `minted` can be used for source code formatting. Note that the `minted` package requires installation of the `Pygments` Python package and also requires the addition of some specialized `arara` commands at the top of the main source code file. Please contact the editors if you need to use the `minted` package and we will figure out which commands need to be added to the main source file.

2.13 Worked Examples

The web interface allows for a user to upload their own example problems (with or without explicit solution) and integrate those with the PDF and/or HTML build. In addition to individual example entry, there is also an option for a *batch import* of multiple examples at once. Note that all examples are visible and available to all registered users; this fits with the open source nature of this project. One important difference: examples and solutions are *not* part of the GitHub repository; they remain private to the web server, so that solutions are not available to web visitors who have not registered on the site. The User Guide, accessible from the navigation bar, describes how to upload an individual worked example, as well as how to prepare a *zip* file for batch import of sets of worked examples. Note that the files *statement.tex* and *solution.tex* are brief source files without any preamble; they should contain only the LaTeX code for the problem and the solution and should not be surrounded by any environment at all. If there is a figure as part of the problem statement or solution, then the following code should be used to include it:

```
\includegraphics[width=0.8\textwidth]{filename.ext}
```

where the extension can be *.pdf*, *.png*, or *.jpg*/*.jpeg*. No figure environment should be used at all.